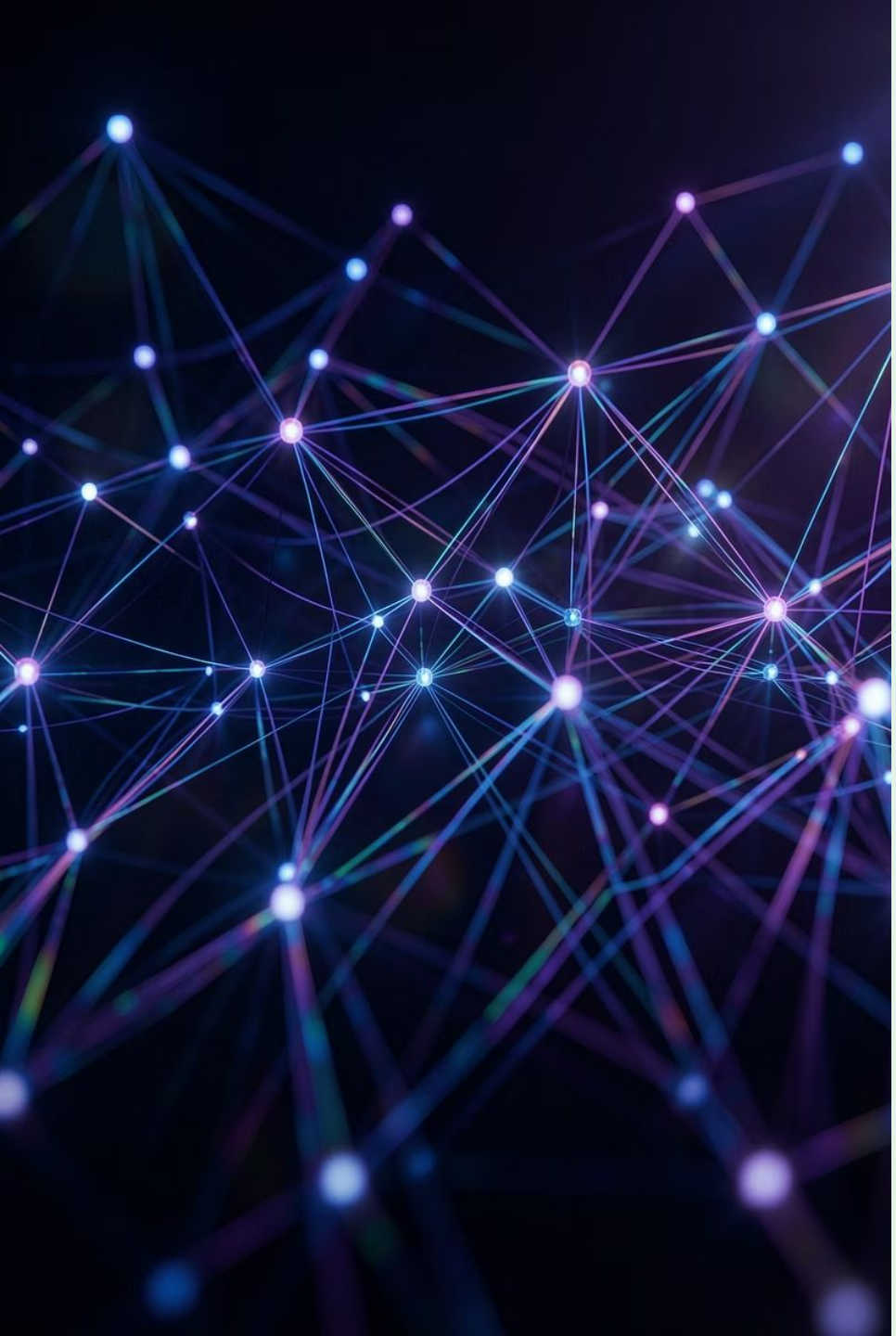


MESA - AI and its Application

Connect, Test, Deploy

21 September 2026 · Cohort C4

Recap - Claude's Core Features

Chat & Cowork

Your primary interface with Claude. Chat for thinking and drafting. Cowork for working directly inside your files and desktop apps.

Projects

A shared context layer across chats. Files you upload and instructions you set apply to every chat inside the project.

Claude.md

A file that travels with your code project. Claude reads it at the start of every session — so it already knows your stack, your rules, and what not to touch.

Skills

Reusable instructions for a type of work. Set once. Apply across any project or chat. We will build one in Case 1.

Connectors & Plugins

Permissions that let Claude access external tools — Google Drive, Gmail, and more. The building blocks of connected workflows.

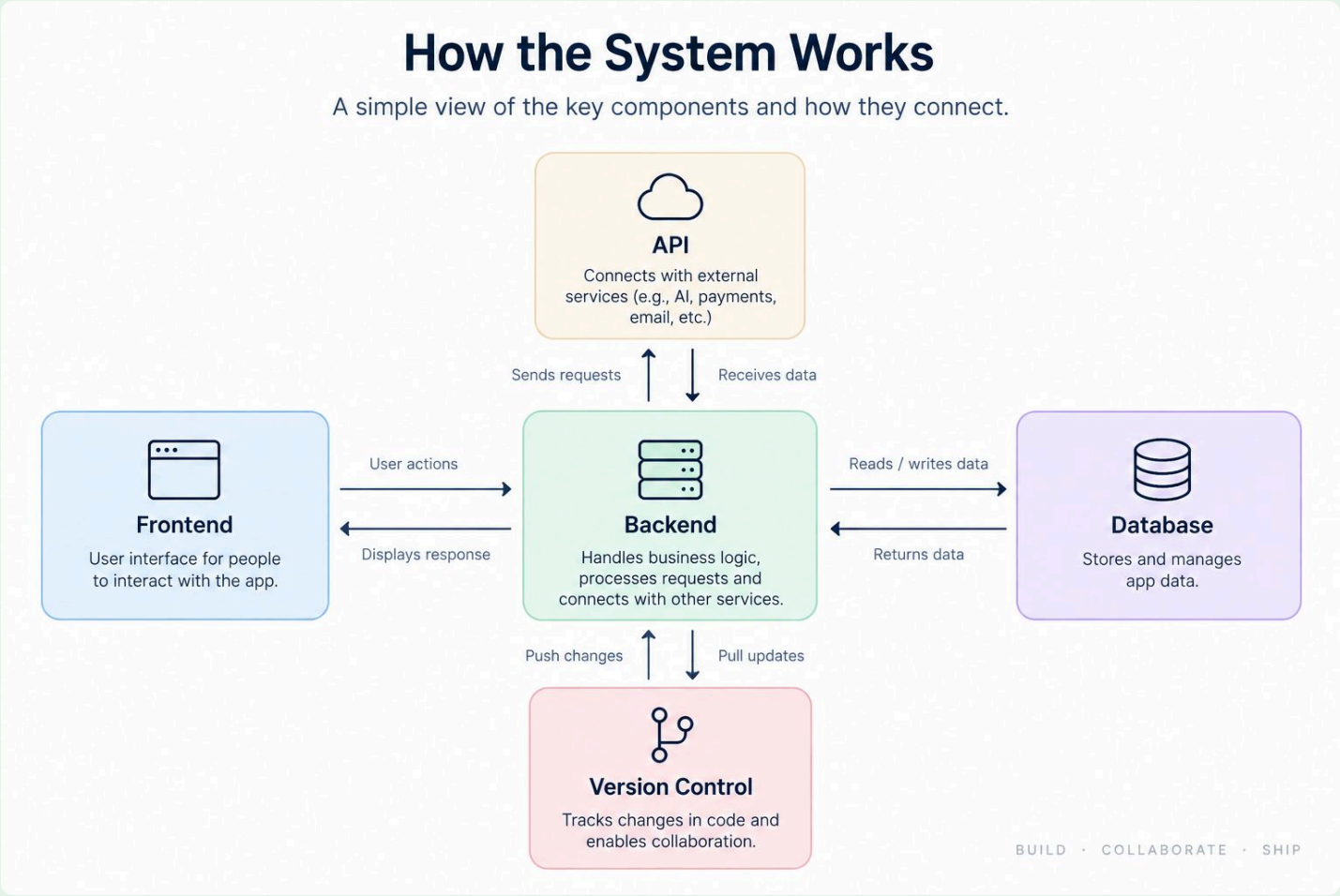
Artifacts

Anything Claude creates that you can keep, download, or share — documents, code, visuals.

The rule of Thumb

Start with the cheapest model that can do the job. Step up only when it visibly fails - not because a better model exists.

The 5 Parts of every App



FlixMatch - What we Built

FlixMatch is a movie matchmaker for two partners who can never agree on what to watch.

Users independently submit their preferences - including mood, genre, language, era, and OTT rating floor. The app then finds 30 titles that suit both, checks which Indian OTT platforms each partner has, and allows both to swipe through cards. When both swipe right on the same title, it's a match, shown instantly on both screens.

Key Integrations Powering FlixMatch

Gemini API

Leveraged to intelligently find the optimal overlap between user preferences.

TMDB

Fetches rich, real-time movie data, including posters and detailed metadata.

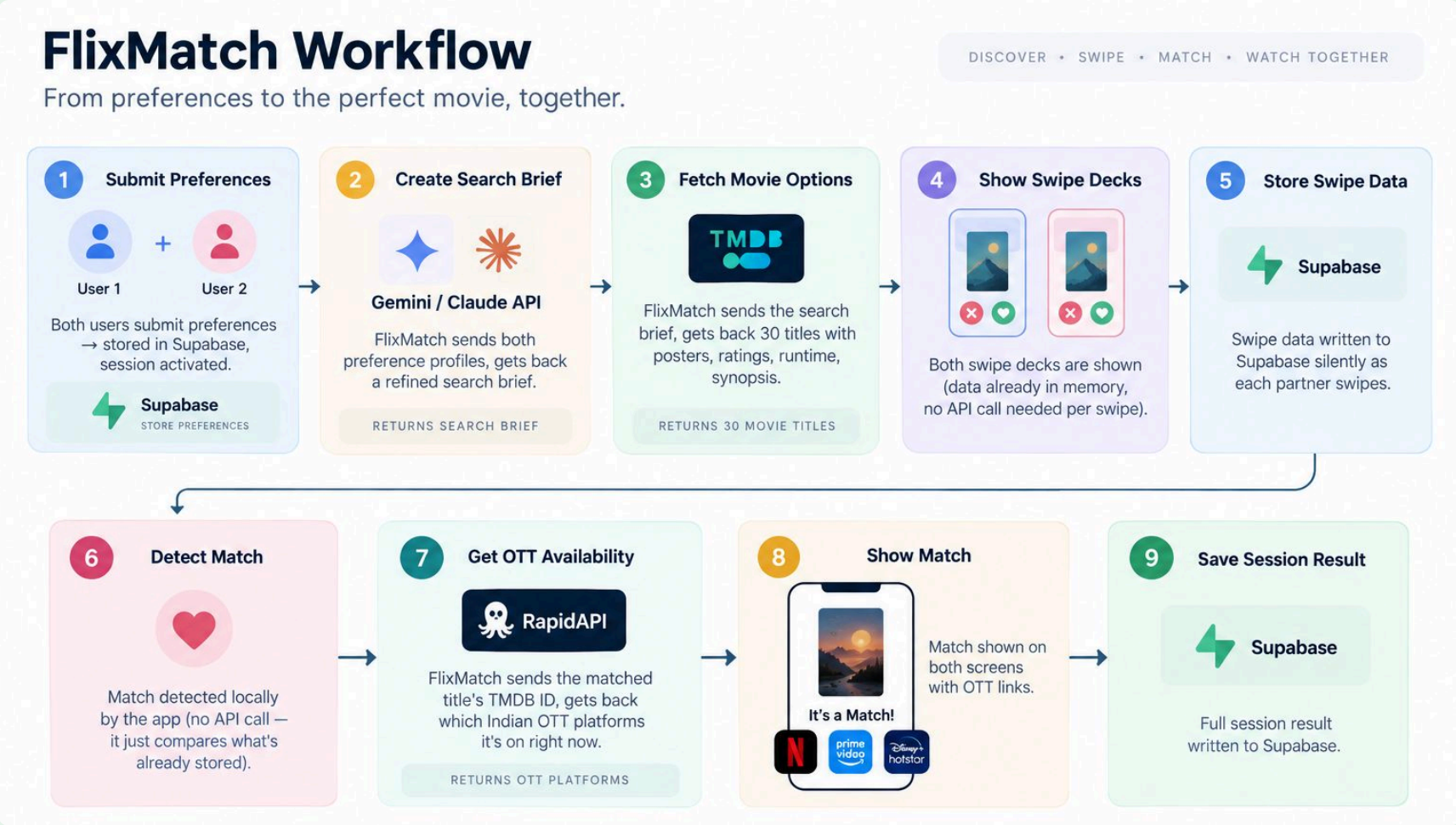
RapidAPI

Confirms live streaming availability across various Indian OTT platforms.

Supabase

Securely remembers every session and user preferences for a seamless experience.

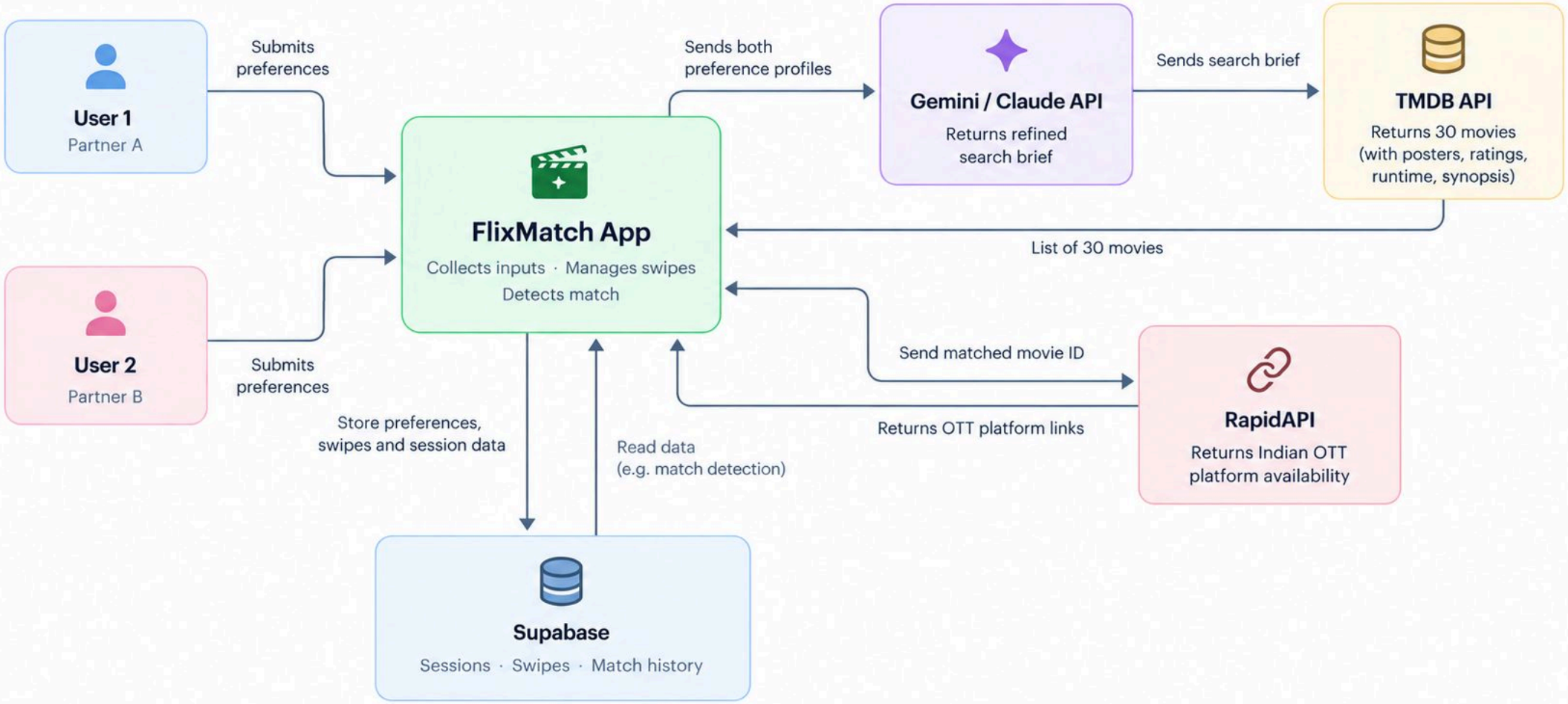
FlixMatch: API Call Flow



FlixMatch — System Workflow

Two people. One movie. A simpler way to decide.

DISCOVER · SWIPE · MATCH · WATCH TOGETHER



Where we left off, and What we do next

First Build - *Complete*

✓ Entered the prompt into Claude Code

✓ Claude built the full application - frontend, backend, swiping mechanic, match logic

Connect Supabase & test app

- Verify the database stores session data.
- Verify the frontend loads and runs.

Deploy on Vercel

- One URL. Live on the internet.

1 2 3 4 5

Add all API keys to the .env file

- Gemini API
- Supabase APIs (URL + Secret Key)
- TMDB API
- RapidAPI

Push to GitHub

- Every change saved.
- Build session starts from something real.

API Keys: Your App's Passwords

When your app wants to use Claude, Supabase, or Telegram, those services need to know: who is asking? The answer is an API key — a unique password that identifies your app and grants it access.

Think of it Like a Hotel Key Card

- The hotel doesn't let anyone walk into any room
- You get a key card at check-in — it only works for your room
- If you lose it, report it and get a new one immediately

Your API key works the same way. If someone else gets it — they walk in as you, use your account, and you get the bill.

> Lose one carelessly — someone else pays for their work using your name and your account. We'll make sure it doesn't happen to you.

The .env File — Where Keys Live

Not in your code. Not in a message. Not pasted into a prompt. In one place — a file called `.env`

Think of it Like a Hotel Safe

- Everything valuable goes in the safe
- Not visible to anyone walking past
- Never packed into your suitcase — it stays in the room

Your .env file is that safe. It holds all your keys and never leaves your computer — ever.

How It Works

```
CLAUDE_API_KEY=sk-ant-...
SUPABASE_KEY=eyJ...
TELEGRAM_TOKEN=7391...
```

Your .gitignore file tells GitHub: "Don't touch this. Don't upload this. Don't show this to anyone." — the .env file is always listed there.

> "Claude Code will remind you to set this up. Always say yes. If it doesn't — you remind it."

Three Rules. No Exceptions.

You don't need to understand everything about security today. You need these three habits - starting now.

1 Keys go in .env

Not in your code. Not in a chat. Not in a shared doc. In `.env` only.

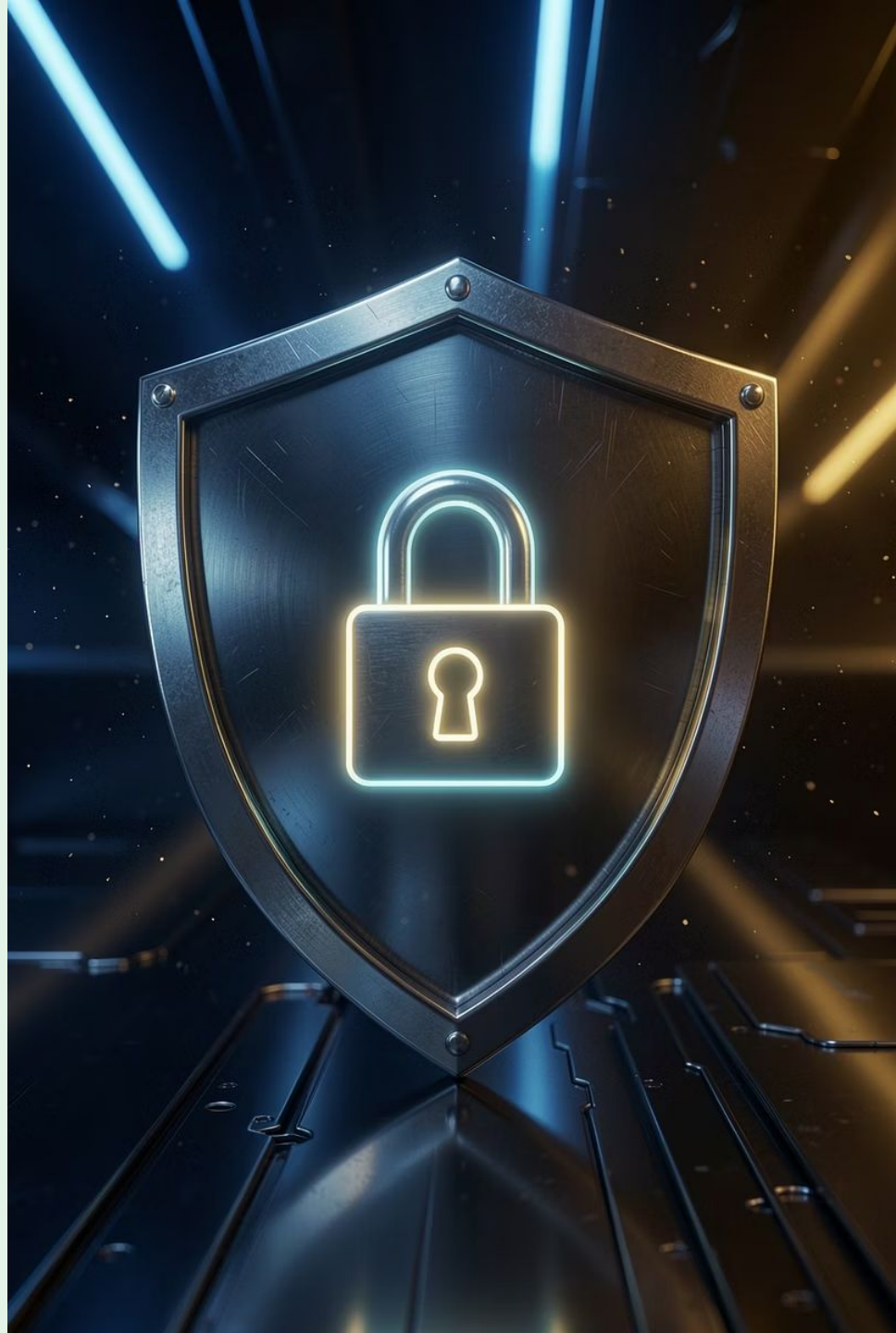
2 .env goes in .gitignore

Before your first GitHub push. If it's not listed, your keys are visible to the entire internet the moment you push.

3 If a key leaks - rotate it immediately

Not tomorrow. Every service lets you delete and regenerate a key in under a minute.

> "A developer pushed code with an AWS key visible. Four hours later — the bill was $50,000. These three rules exist so that never happens to you."





Your App: Local Only

FlixMatch ran solely on your laptop, which means:

- Only works while your laptop is running.
- Only you can open it.
- Friends can't access the URL.

> How do you take an app from your laptop and make it globally accessible?
>
> **The answer: A server.**

A SERVER IS JUST SOMEONE ELSE'S COMPUTER

- one that never turns off

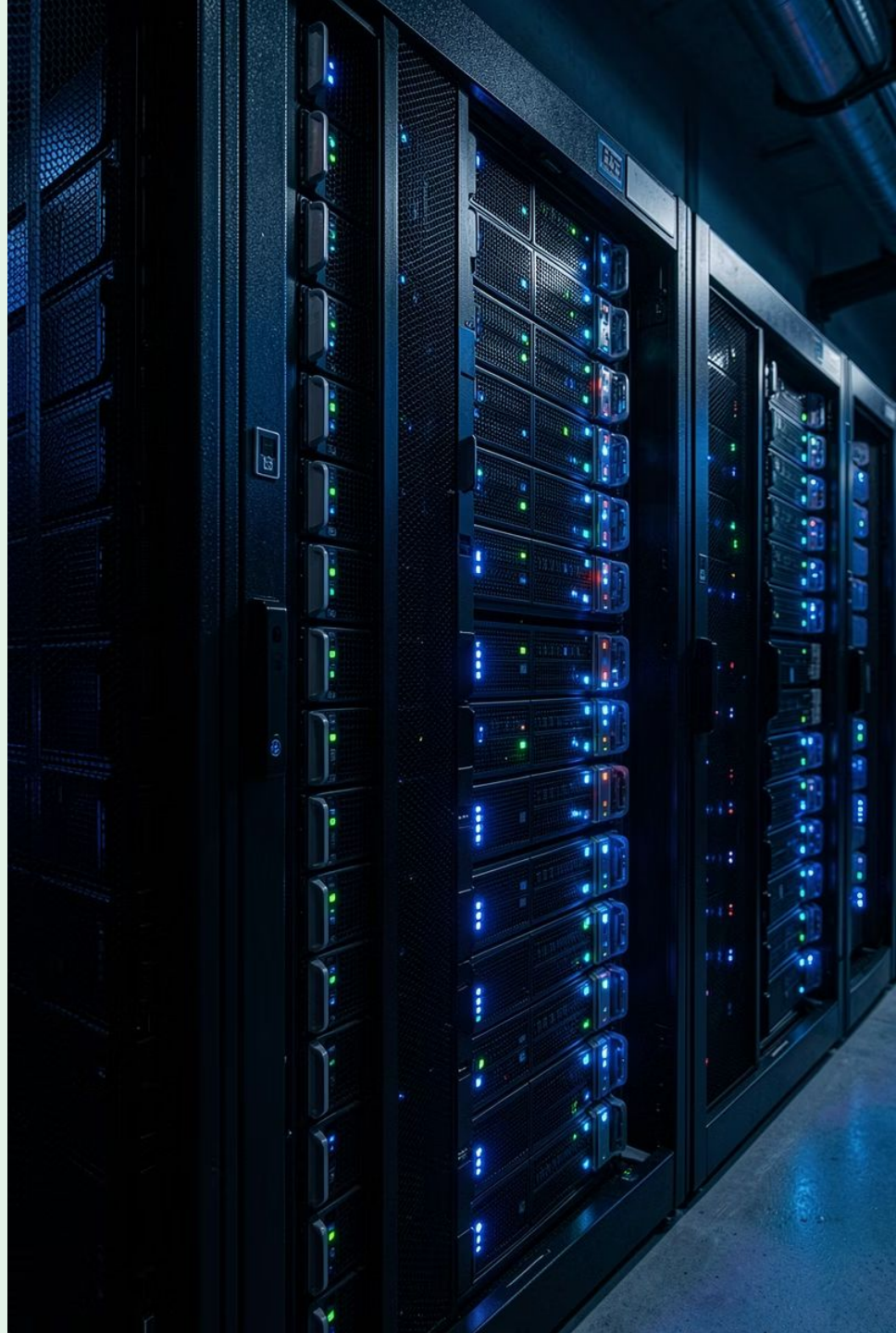
When you open Netflix, your phone sends a request to a server, and that server sends your show back - 24/7, without fail.

Your app needs the same setup: a computer that's always on, always connected, and ready to respond to anyone who visits your URL.

You don't own this computer - you rent it from a **hosting provider.**

Vercel is that hosting provider.

They provide the server. You provide the code.



What Vercel Does

After you hit deploy, Vercel takes your code through four steps to go live.

01

Reads Code from GitHub

Vercel pulls your latest code directly from your GitHub repository.

02

Builds Your Application

Dependencies are installed and your code is compiled into browser-readable format.

03

Distributes Globally

Your app is replicated across Vercel's global Edge Network for fast access.

04

Assigns a Unique URL

Vercel gives your app a permanent URL, live and accessible worldwide.

YOUR CODE → GITHUB → VERCEL → THE INTERNET

Every push to GitHub triggers this process, updating your live app automatically.

Three Steps to a Live URL

No commands. No terminal. Just three steps.

1 Connect your GitHub repo to Vercel

Open Vercel. Sign in with GitHub. Import the FlixMatch repository.

2 Add your environment variables

Every key from your `.env` file goes into Vercel's dashboard manually. Your `.env` file never leaves your laptop. Vercel needs its own copy.

3 Deploy

One click. Vercel builds your app. In under two minutes — you have a live URL. Open that URL on your phone. FlixMatch is on the internet.



Why ENV Variables Appear Again Here

Your `.env` file keeps keys off GitHub. But Vercel's servers need those keys too - so you add them manually in Vercel's dashboard.

Your `.env` file

Local development only

Vercel dashboard

For the live app

Nowhere else. Ever.

This is the rule.

Do This Now

01

Tell Claude Code

`"Connect this project to GitHub and push my latest code."`

02

Open Vercel

vercel.com → Sign in with GitHub → Import Repository → FlixMatch

03

Add environment variables

Add every key in the Vercel dashboard.

04

Deploy

Your live URL appears when the build finishes.

Do this now - follow each step in order.